

## Program for 2d Array

```
int matrix[4][4];
int value = 1;

for(int i=0; i<4; i++){
    for(int j=0; j<4; j++){
        matrix[i][j] = value;
        value++;
    }
}
```

```
.data
matrix: .space 64    # 4 x 4 x 4 bytes = 64 bytes
```

```
.text
.globl main
```

main:

```
la $s0, matrix    # base address of matrix
li $t0, 0          # i = 0 (row)
li $t4, 4          # number of columns = 4
li $t7, 1          # value to store
```

outer\_loop:

```
bge $t0, 4, exit  # if i >= 4 -> exit
li $t1, 0          # j = 0 (column)
```

inner\_loop:

```
bge $t1, 4, next_row # if j >= 4 -> next row
```

# offset = (i \* 4 + j) \* 4

```
mul $t2, $t0, $t4  # i * 4
add $t2, $t2, $t1  # i*4 + j
sll $t2, $t2, 2    # multiply by 4 (int size)
```

```
add $t3, $s0, $t2  # address = base + offset
```

```
sw $t7, 0($t3)    # store value
```

```
addi $t7, $t7, 1  # value++
```

```

        addi $t1, $t1, 1    # j++
        j    inner_loop

next_row:
        addi $t0, $t0, 1    # i++
        j    outer_loop

exit:
        li   $v0, 10
        syscall

```

```

int matrix[16][16];
int value = 0;

```

```

for(int i = 0; i < 16; i++) {
    for(int j = 0; j < 16; j++) {
        matrix[i][j] = value;
        value++;
    }
}

```

***How is it storing....??***

```

Row 0: 0  1  2  3  ... 15
Row 1: 16 17 18 19  ... 31
Row 2: 32 33 34 35  ... 47
...

```

- **Row-major.asm**
- **Traverses a 16x16 matrix in row-major order**
- **Assigns values 0 through 255 sequentially**

.data

matrix: .space 1024      # 16 x 16 x 4 bytes = 1024 bytes

.text

.globl main

main:

```

    la  $s0, matrix    # $s0 = base address of matrix
    li  $t0, 0          # i = 0 (row index)
    li  $t4, 16         # constant: number of columns = 16
    li  $t7, 0          # value to store (0 to 255)

```

outer\_loop:

```

    bge $t0, 16, exit   # if i >= 16 ? exit
    li  $t1, 0          # j = 0 (column index)

```

inner\_loop:

```

    bge $t1, 16, next_row # if j >= 16 ? go to next row

```

# Row-major address calculation:

# offset =  $(i * 16 + j) * 4$

mul \$t2, \$t0, \$t4    #  $i * 16$

add \$t2, \$t2, \$t1    #  $i*16 + j$

sll \$t2, \$t2, 2      # multiply by 4 (shift left 2 bits)

add \$t3, \$s0, \$t2    # final address

sw \$t7, 0(\$t3)      # store current value

addi \$t7, \$t7, 1     # value++

addi \$t1, \$t1, 1     # j++

j inner\_loop

next\_row:

addi \$t0, \$t0, 1     # i++

j outer\_loop

exit:

li \$v0, 10

syscall

- **column-major.asm**
- **Traverses a 16x16 matrix in column-major order**
- **Assigns values 0 through 255 sequentially**

.data

matrix: .space 1024      # 16 x 16 x 4 bytes = 1024 bytes

.text

.globl main

main:

la \$s0, matrix      # \$s0 = base address of matrix

li \$t0, 0            # j = 0 (column index)

li \$t4, 16           # constant size = 16

li \$t7, 0            # value = 0

outer\_loop:

bge \$t0, 16, exit    # if j >= 16 ? exit

li \$t1, 0            # i = 0 (row index)

inner\_loop:

bge \$t1, 16, next\_col # if i >= 16 ? next column

# Column-major address calculation:

# offset = (i \* 16 + j) \* 4

# BUT traversal order is column-first

mul \$t2, \$t1, \$t4    # i \* 16

add \$t2, \$t2, \$t0    # i\*16 + j

sll \$t2, \$t2, 2      # multiply by 4 (word size)

```
add $t3, $s0, $t2    # final address
```

```
sw  $t7, 0($t3)      # store value
```

```
addi $t7, $t7, 1      # value++
```

```
addi $t1, $t1, 1      # i++
```

```
j    inner_loop
```

```
next_col:
```

```
    addi $t0, $t0, 1    # j++
```

```
    j    outer_loop
```

```
exit:
```

```
    li $v0, 10
```

```
    syscall
```

# Assignment: Effect of Block Size on Cache Hit Rate

## Title

Analysis of Cache Hit Rate Variation with Increasing Block Size

---

## Objective

To study how increasing block size affects cache hit and miss rates for different memory mapping techniques.

---

## Problem Statement

You are given a MIPS program that performs matrix traversal (row-major traversal recommended).

Your task is to analyze how cache hit rate changes when block size increases from very small to larger values.

You must perform the experiment using:

- Direct-Mapped Cache
- 2-Way Set Associative Cache
- Fully Associative Cache

---

## Experimental Design

Keep the following constant:

- Word size: 4 bytes
- Cache size: 1024 bytes
- Program: Same matrix traversal code

Now vary **block size** progressively:

- 8 bytes
- 16 bytes
- 32 bytes
- 64 bytes
- 128 bytes

(If simulator allows other sizes, include them.)

---

## Tasks

For each mapping technique:

1. Start with the smallest block size.
2. Run the program.
3. Record:
  - Total memory accesses
  - Cache hits
  - Cache misses
  - Hit rate (%)
4. Increase block size step by step.
5. Continue until hit rate stops improving or reaches maximum.

---

### Observation Table Format

| Mapping Type | Block Size (Bytes) | Hits | Misses | Hit Rate (%) |
|--------------|--------------------|------|--------|--------------|
| Direct       | 8                  |      |        |              |
| Direct       | 16                 |      |        |              |
| Direct       | 32                 |      |        |              |
| Direct       | 64                 |      |        |              |
| Direct       | 128                |      |        |              |

Repeat for Set-Associative and Fully Associative.